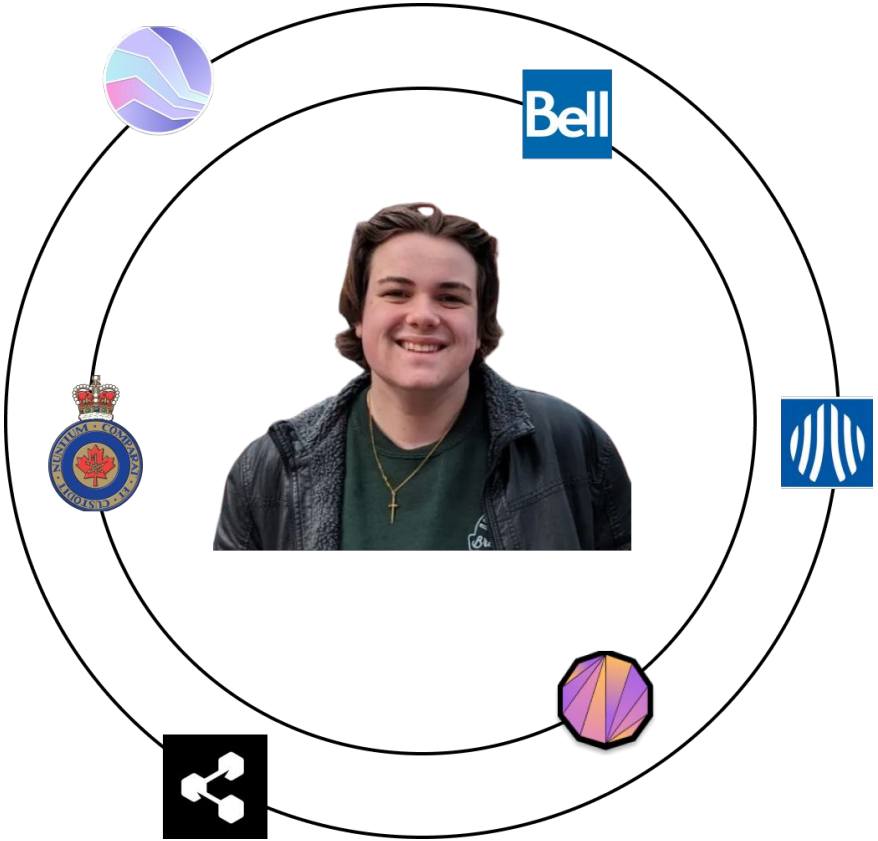
Rate Limiting: The Art of Saying No

by: TJ Klint

My Name is TJ

- Software Developer @ Botpress
- Startups, Telecom, Government, Freelance, and More
- Building tech communities



1. Why Rate Limiting Exists

2. The Algorithms.

3. The Hard Part

4. Conclusion

Section 1: Why Rate Limiting Exists

Shmistopher Shrass

Freelance Security, RL Co.

Rules:

- Max 10 people per minute
- No exceptions

March 23, 2010

Shmoss Shiag
Head of Engineering @ The White House

EVERY SINGLE AMERICAN:

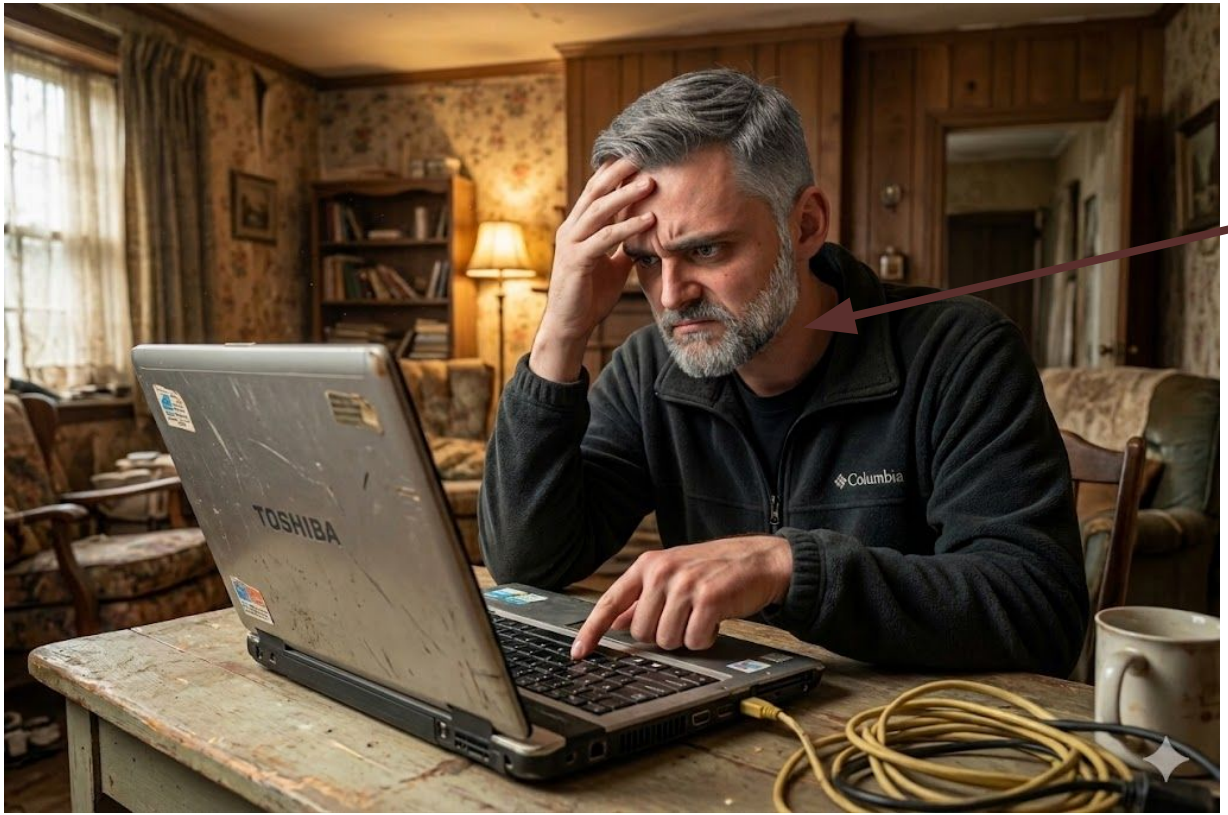
Go to healthcare.gov, check your eligibility, and enroll for FREE HEALTHCARE.

Shmoss Shiag
Head of Engineering @ The White House

EVERY SINGLE AMERICAN:

Go to healthcare.gov, check your eligibility, and enroll for FREE HEALTHCARE.

BINGO HALL
OPEN
OPEN
BINGO
BINGO
BINGO
BINGO
BINGO
Columbia



Shmancois Shavasseur
old grandpa.
Lafayette, LA

Status:

- Clearly not happy
- Spamming refresh
- Old as hell.

A Few Puddings Later

A real screenshot that millions of americans viewed for days...

HealthCare.gov

Learn | Get Insurance | Log in | Español

Individuals & Families | Small Businesses | All Topics

Search | SEARCH

The System is down at the moment.

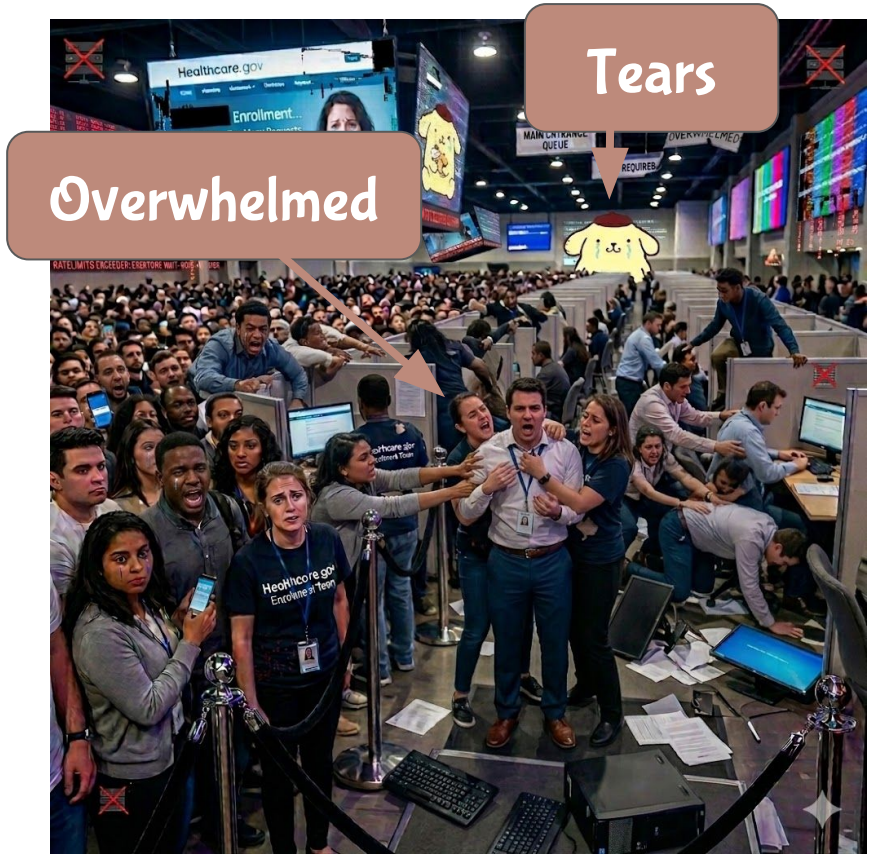
We're working to resolve the issue as soon as possible. Please try again later.

HealthCare.gov
Center for Health Insurance
and Marketplace Operations
HealthCare.gov
HealthCare.gov
HealthCare.gov
Center for Health Insurance
and Marketplace Operations

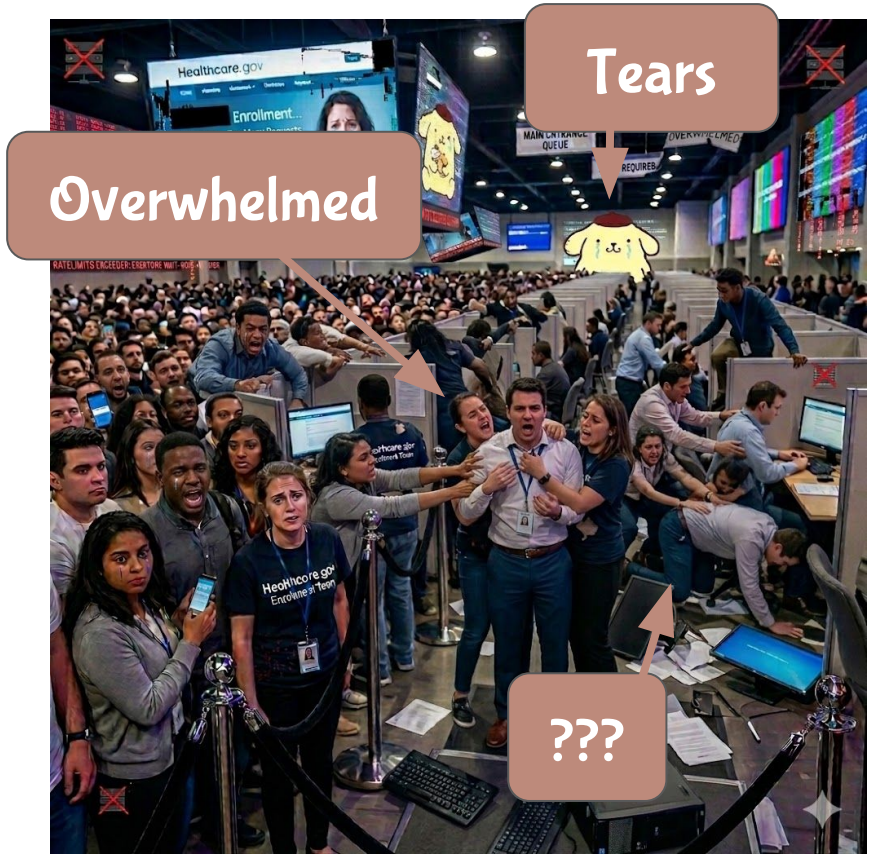
Without Shmistopher:

Without Shmistopher:

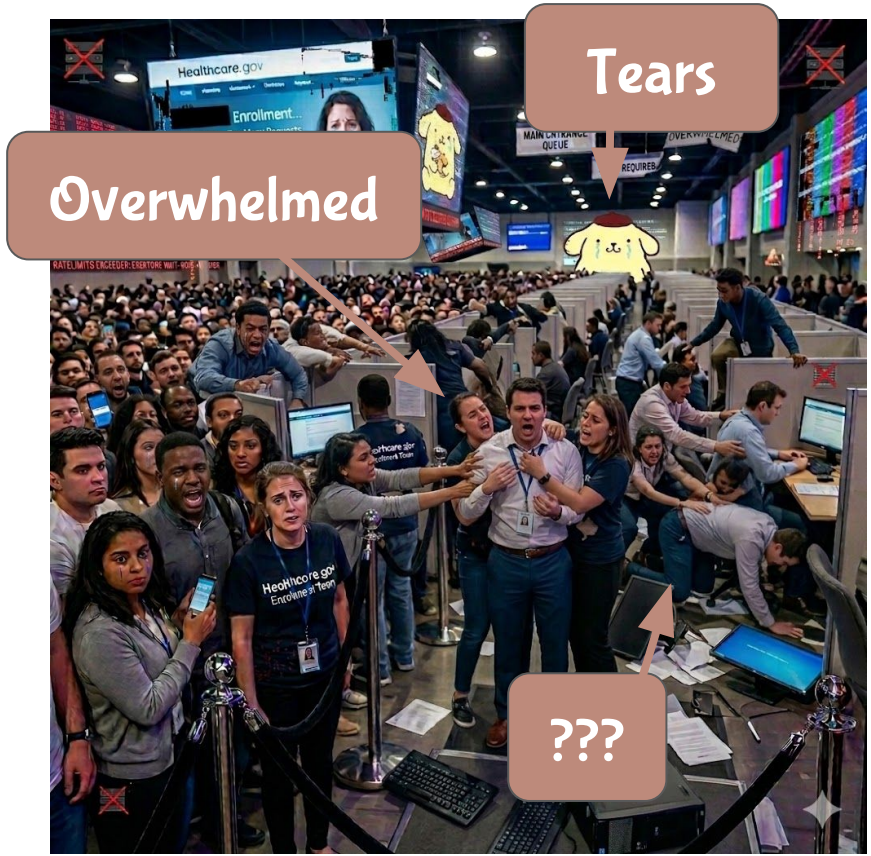
Without Shmistopher:



Without Shmistopher:

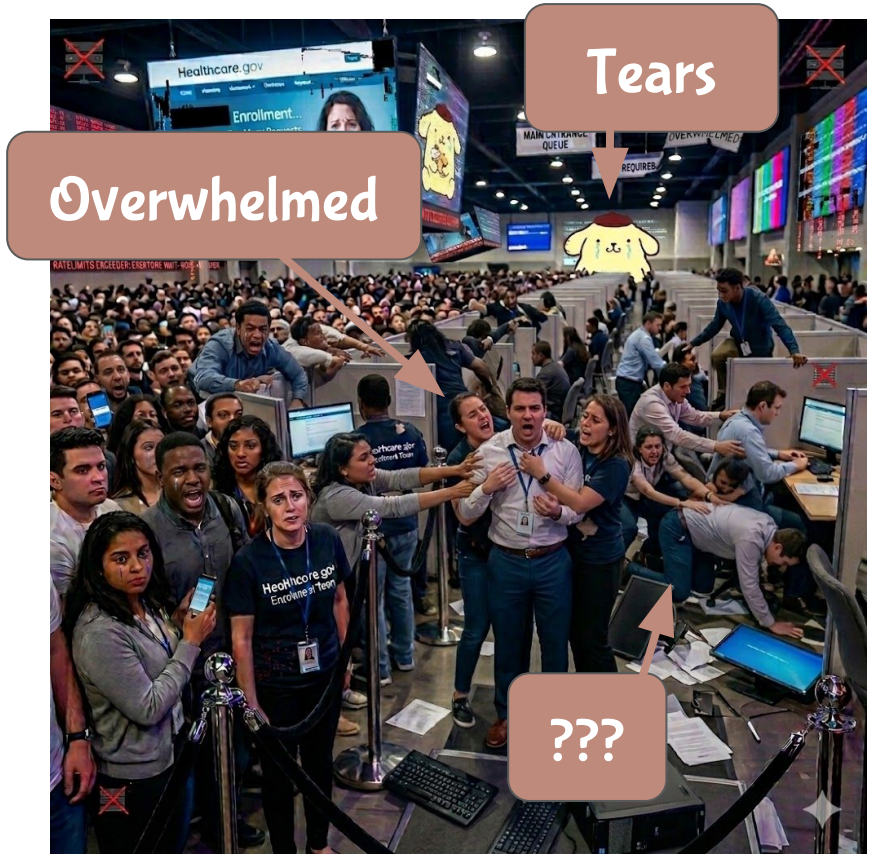


Without Shmistopher:



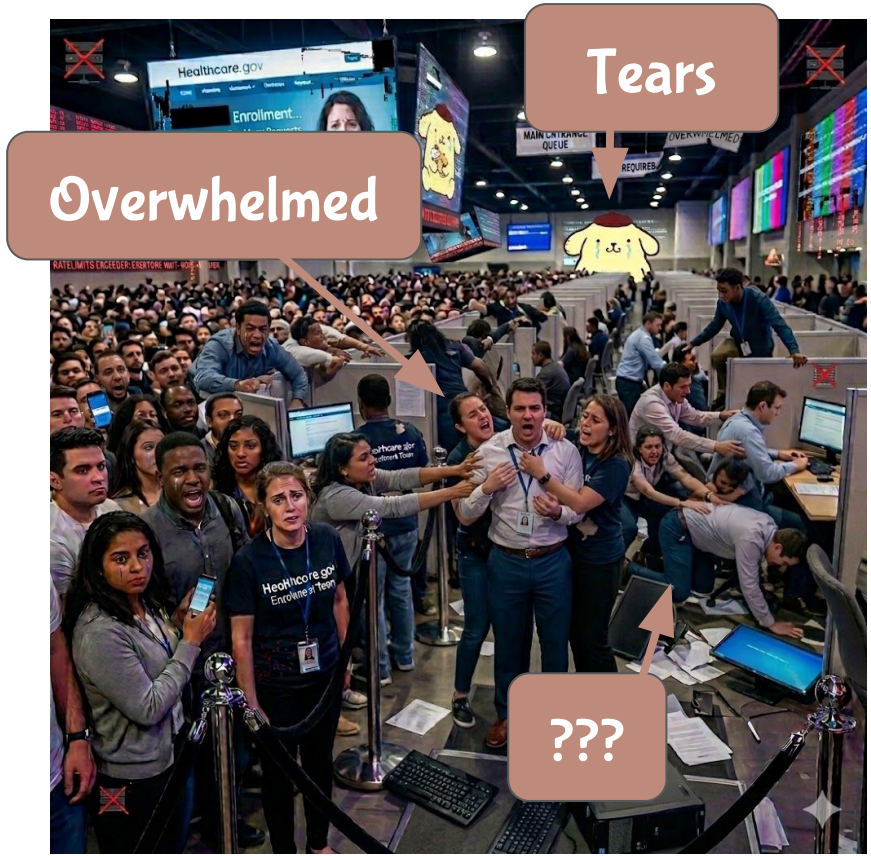
With Shmistopher:

Without Shmistopher:



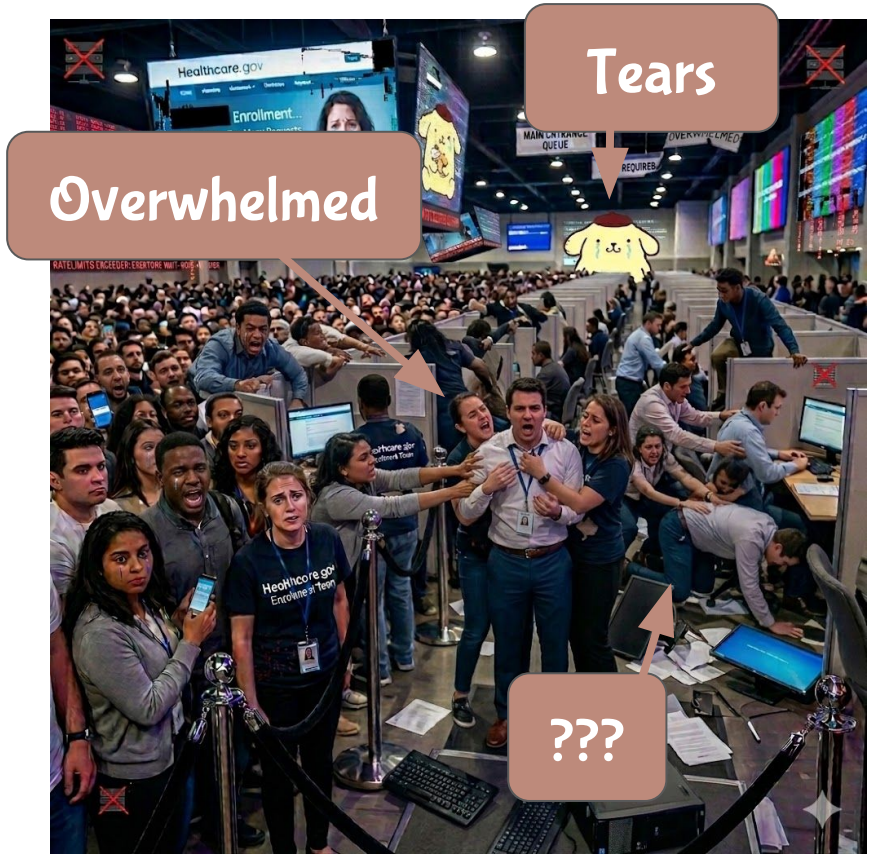
With Shmistopher:

Without Shmistopher:



With Shmistopher:

Without Shmistopher:



With Shmistopher:

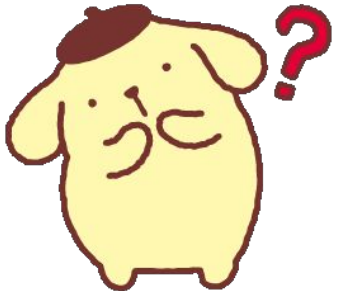
Healthcare.gov didn't just crash because too many people showed up.

It crashed because nobody had thought about what "too many" even meant.

What did healthcare.gov need?

- 🛡️ Protection: The building shouldn't collapse just because it got popular. Someone needs to hold the door.
- ⚖️ Fairness: One person shouldn't be able to hold the door open for themselves while everyone else waits outside.
- 📋 Predictability: If you know Shmistopher lets in 10 people per minute, you can plan around that. Chaos is the enemy of planning.

So how do we do it, and do it right?



Section 2: The Algorithms

Fixed Window

The Idea:

1. Divide time into fixed buckets (e.g. 60 seconds)
2. Each bucket has a counter
3. Request comes in → increment counter
4. Counter hits limit → reject until next window

Fixed Window

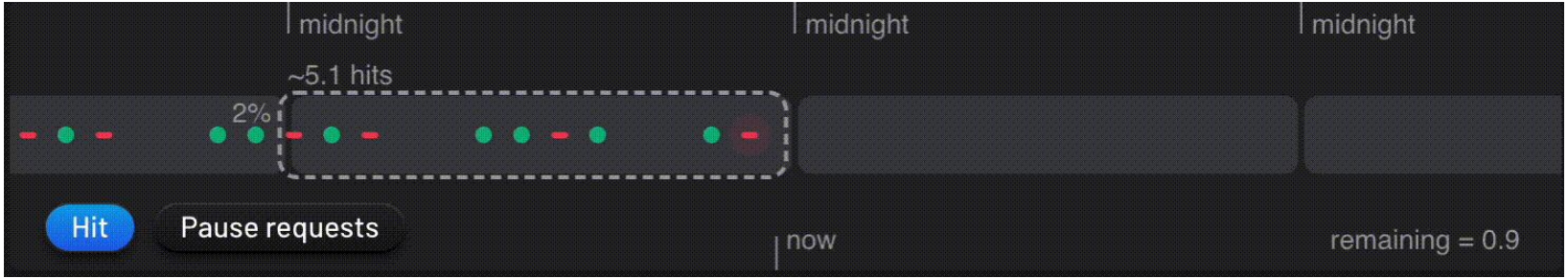
The Idea:

1. Divide time into fixed buckets (e.g. 60 seconds)
2. Each bucket has a counter
3. Request comes in → increment counter
4. Counter hits limit → reject until next window

The Problem:

1. Boundary spike: up to 2x your limit can sneak through at window edges
2. Bursty by design: all 10 requests can hit in the first second, leaving the remaining 59 seconds wide open
3. No memory: the window has no idea what happened before it started

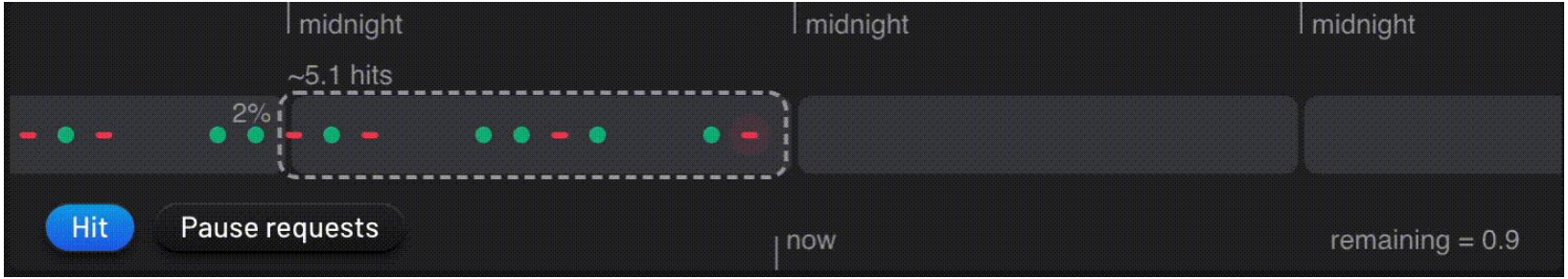
Sliding Window



The Idea:

1. Instead of fixed buckets, track the exact timestamp of every request
2. When a new request comes in, look back exactly 60 seconds
3. Count the requests in that range: if under limit, allow. If over, reject.

Sliding Window



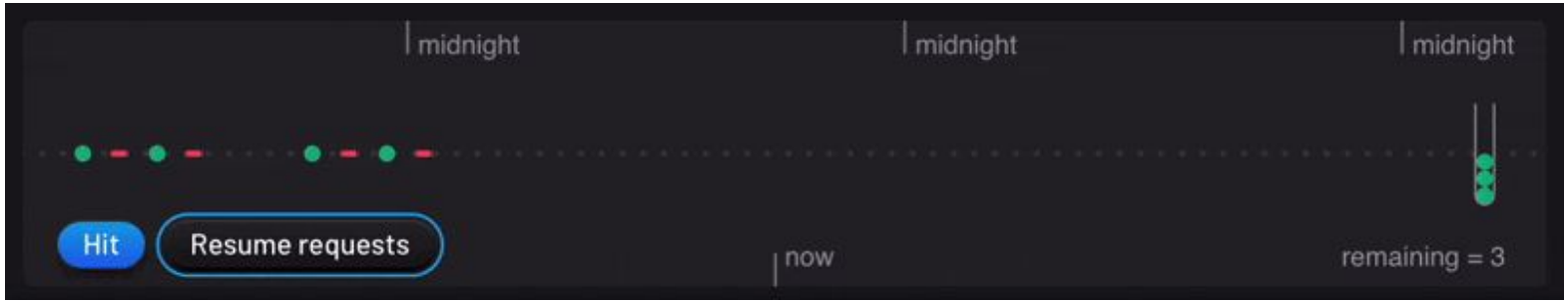
The Idea:

1. Instead of fixed buckets, track the exact timestamp of every request
2. When a new request comes in, look back exactly 60 seconds
3. Count the requests in that range: if under limit, allow. If over, reject.

The Problem:

1. Memory hungry. You're storing every single timestamp, per user, per endpoint
2. Expensive at scale: 1M users x 100 requests = a lot of timestamps to store and query

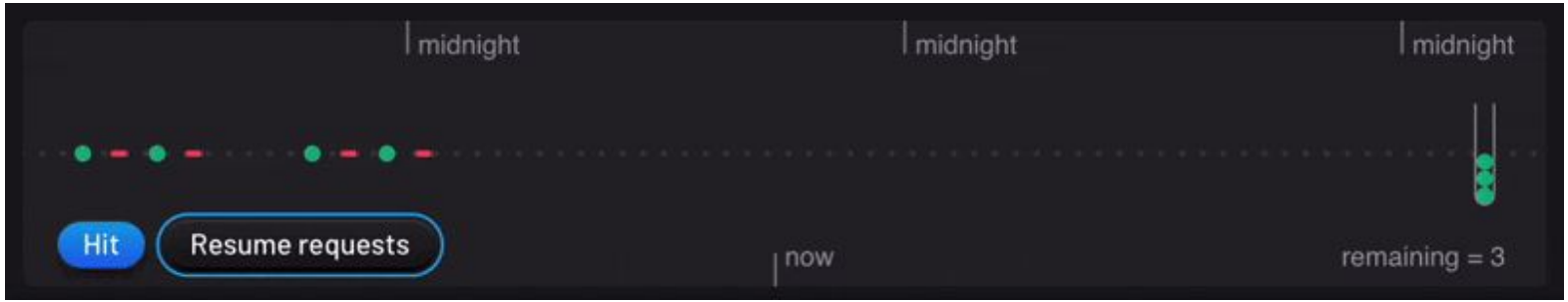
Token Bucket



The Idea:

1. You have a bucket with a max capacity of N tokens
2. Tokens drip in at a fixed rate (e.g. 10/min)
3. Each request costs a token
4. No tokens? Rejected. Bucket full? Excess drips are discarded.

Token Bucket



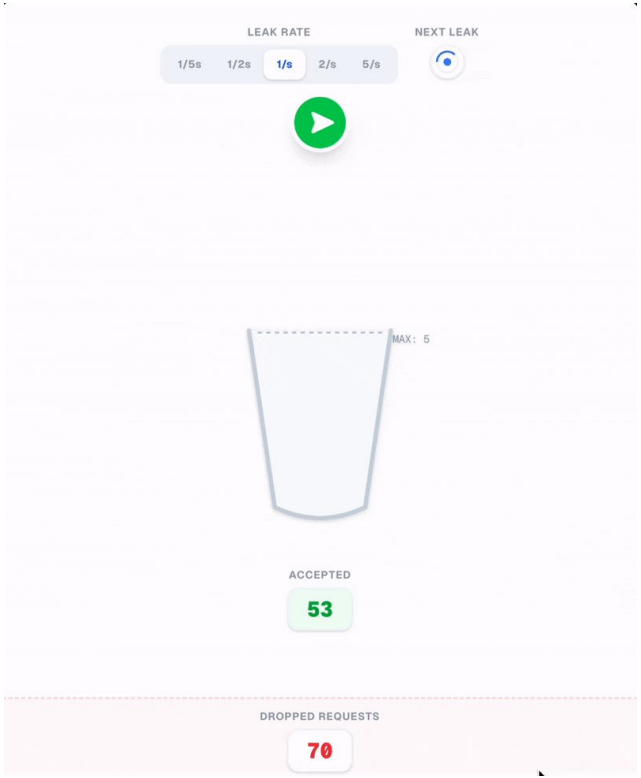
The Idea:

1. You have a bucket with a max capacity of N tokens
2. Tokens drip in at a fixed rate (e.g. 10/min)
3. Each request costs a token
4. No tokens? Rejected. Bucket full? Excess drips are discarded.

The Problem:

1. Burst behavior can still spike your backend if not tuned carefully
2. Two parameters to tune (capacity + refill rate) easy to get wrong
3. Clock drift and race conditions matter a lot here (we'll get there)

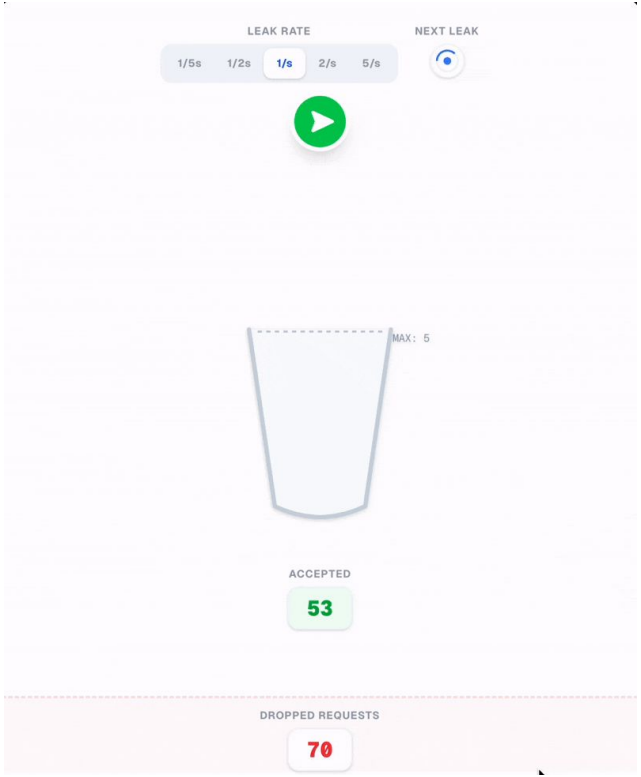
Token Bucket



The Idea:

1. Requests go into a queue (the bucket)
2. They are processed at a fixed, constant rate: no exceptions
3. Bucket full? New requests are dropped.

Leaky Bucket



The Idea:

1. Requests go into a queue (the bucket)
2. They are processed at a fixed, constant rate: no exceptions
3. Bucket full? New requests are dropped.
4. Anywhere downstream can't tolerate spikes: payment processors, third party APIs you're calling

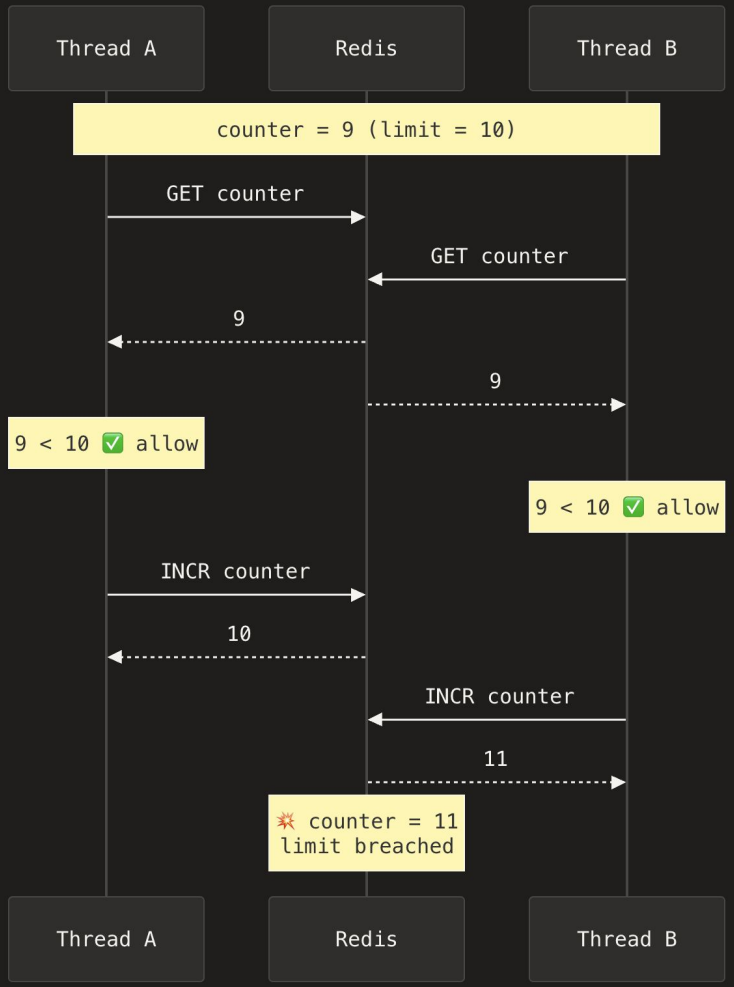
The Problem:

1. Bursts get flattened even if your system could handle them
2. Queue adds latency: requests wait instead of being rejected fast
3. Poor fit for user-facing APIs where fast rejection is better than slow processing

Section 3: The Hard Part

Race Conditions

- Both threads read the same value before either writes
- Neither knows the other exists
- You've exceeded your own limit with fully "correct" code



Use Lua!

Redis is single-threaded, and Lua scripts execute on the server, blocking all other commands until completion.

```lua
local current = redis.call('INCR', KEYS[1])
if current == 1 then
  redis.call('EXPIRE', KEYS[1], ARGV[1])
end
if current > tonumber(ARGV[2]) then
  return 0
end
return 1
```

Luckily, you can just use these.

rate-limiter-flexible TS

11.0.0 • Public • Published 10 days ago

JS TS

limits 5.8.0

```
pip install limits
```

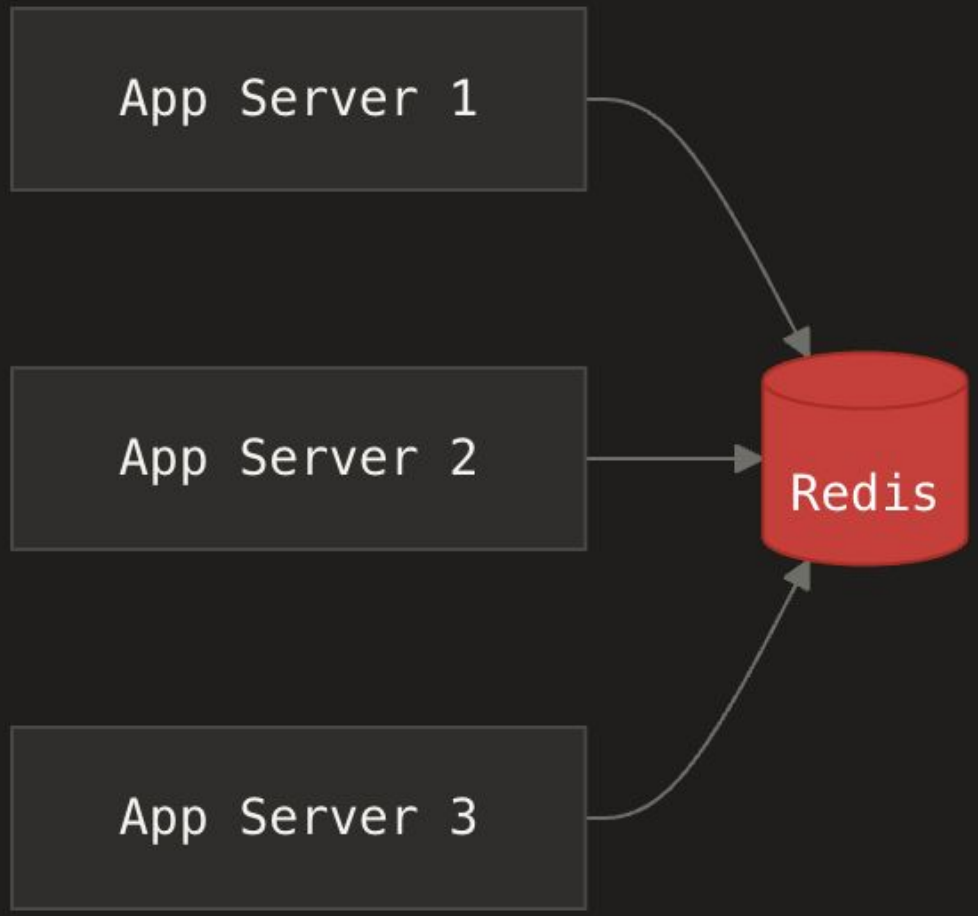
.NET **System.Threading.RateLimiting** 10.0.5

Prefix Reserved

.NET 8.0 | .NET Standard 2.0 | .NET Framework 4.6.2

Time to choose a Redis Topology...

Not too hard of a choice, however, the implications can be huge on your application.

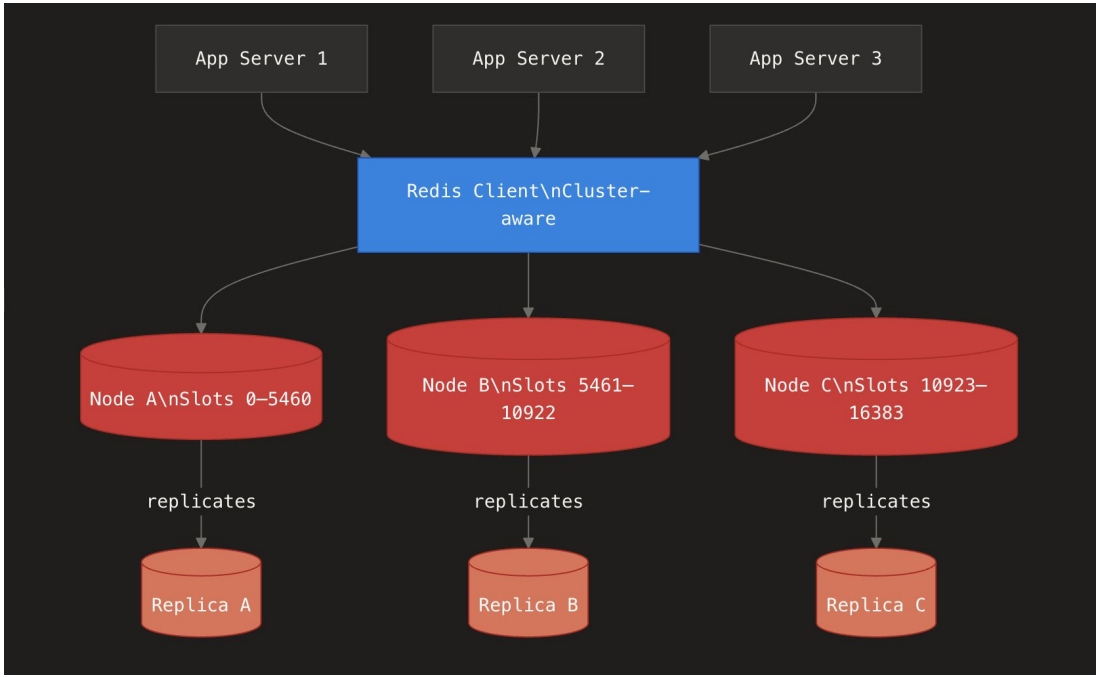


Single Node

One Redis. All your app servers point at it. Simple, fast, and atomic Lua scripts work perfectly there's nothing to coordinate.

- Atomic operations work exactly as expected, no gotchas
- If Redis goes down, your rate limiter goes down with it
- No replication lag, no split-brain, no slot routing. WYSIWYG.
- Fine for most use cases until you need HA or your single node becomes a bottleneck

Cluster

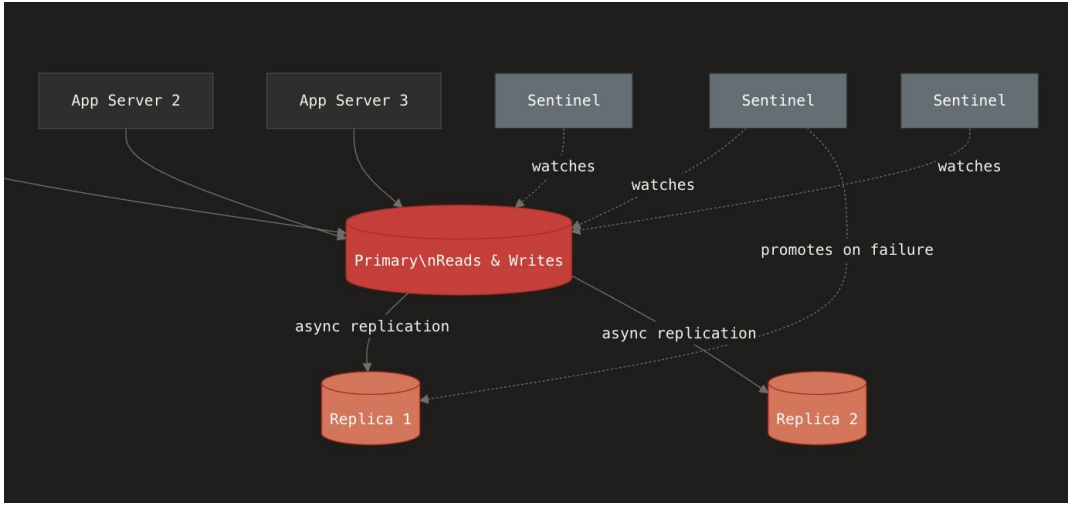


Cluster shards your keyspace across multiple nodes using 16,384 hash slots. Each node owns a slice, each node has a replica.

- Lua scripts must operate on keys that live on the same node. If they don't, the script fails
- Fix: hash tags, {user:123}:counter forces all keys for a user to the same slot
- If a node goes down, its slice of keyspace is unavailable during promotion: same failover dilemma as sentinel, but scoped to a subset of your keys
- Higher complexity: make sure your Redis client is cluster-aware or you'll get silent misrouting

Sentinel

Sentinel adds automatic failover. Three sentinels watch the primary, hold a vote, and promote a replica if the primary dies.



- Replication is async, the promoted replica may be a few writes behind
- Those missing writes are missing increments, you may let more requests through than you should during the failover window
- Failover takes 15-30 seconds.
- Most teams run this thinking they have full HA. they do, but with a gap they haven't thought about

What happens when Redis dies...?

Fail-open

- Users stay unaffected
- Your backend is exposed for the entire outage window
- **Exploitable**: sophisticated attackers know infrastructure blips happen

Fail-closed

- Backend stays protected
- You're issuing 429s due to your own infrastructure failure, not abuse
- Can cascade: if rate limiting is in your hot path, a Redis outage becomes a full outage

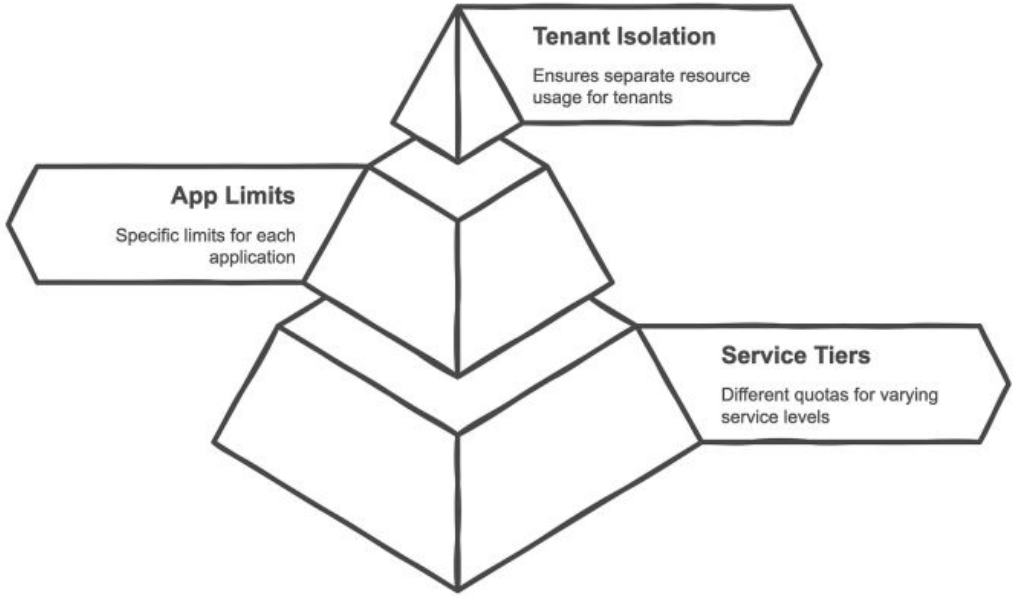
What happens when Redis dies...?

Circuit Breaker!

Decision	Question	Answer
Failure threshold	What trips the breaker?	Error rate >50% over 10s — not raw count, which is volume-sensitive
Timeout	How long do you stay Open?	Start at 5s, back off exponentially on repeated failures
Probe strategy	How do you test recovery?	Single request — but don't let a flaky node flip you back prematurely
Fallback	What runs while Open?	Pre-tuned in-memory limit — know your blast radius before the outage
Breaker scope	One breaker or many?	Per dependency — never share across rate limit Redis, cache, sessions

Hierarchical rate limiting

One limit is never enough. Real systems enforce multiple limits at different layers simultaneously.



Pro tip for enterprise applications:
Service: protects infrastructure regardless of who's asking
Tenant: one large customer can't starve others in a multi-tenant system
User: one rogue user can't exhaust a tenant's quota
Endpoint: a cheap endpoint shouldn't be used to bypass limits on an expensive one

NOTE: Consider whether you can pipeline them, or whether some layers can be checked probabilistically rather than on every single request.

Be smart with your keys.

```
ip:192.168.1.1
user:usr_123
apikey:key_abc
tenant:acme
tenant:acme:user:usr_123          ← composite
tenant:acme:endpoint:/v1/messages ← composite

// ALWAYS include service and environment in the key
payments:prod:user:usr_123
payments:staging:user:usr_123
```

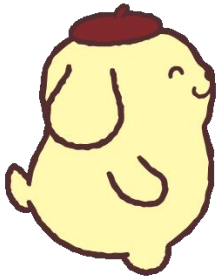
TTL (time to live): set it on creation, always:

1. No TTL = keys accumulate forever
2. TTL too short = counter resets mid-window
3. TTL should equal your window duration, set atomically on first write.

Cardinality

starts simple, gets dangerous

```
tenant:endpoint:user
```



This sounds reasonable.

At 10,000 tenants × 50 endpoints × 10,000 users = 5 billion possible keys.

Most are cold, but Redis still has to manage TTLs and memory for every live one.

Design your key hierarchy to match your actual access patterns, not your theoretical ones. Drop the endpoint dimension if per-endpoint per-user limits aren't a real requirement.

Hotspot Keys

A global rate limit counter is incremented by every request across every app server.

Redis is single-threaded.

At high enough throughput, think 50k-100k req/s: that single key becomes a serialization bottleneck.

Every increment queues behind the last one. You've built a mutex into your hot path.

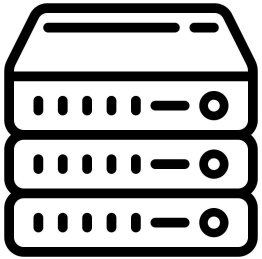
Counter Sharding

```
1  global:counter:shard:0
2  global:counter:shard:1
3  ...
4  global:counter:shard:N
```

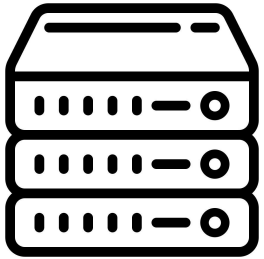
```python
# Write path — random shard
shard = random.randint(0, NUM_SHARDS - 1)
redis.incr(f"global:counter:shard:{shard}")

# Read path — periodic aggregation
total = sum(redis.get(f"global:counter:shard:{i}") for i in range(NUM_SHARDS))
cache.set("global:counter:total", total, ttl=1)

```

Clock Drift

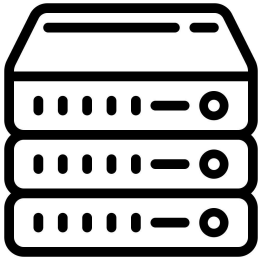
In a distributed system, every server has its own system clock, and those clocks are never perfectly in sync. They drift apart over time: typically by milliseconds, sometimes more.



12:45:22:293



12:45:22:353



12:45:22:642

Solution: Use Redis as your single source of time truth. Instead of relying on your app server clocks, use Redis's TIME command to get the current timestamp

Section 4: Conclusion

UX (coming from a backend engineer :D)

Paid SaaS

Your users are paying for a quota. They have a right to know how much they've used, how much they have left, and when it resets

```
X-RateLimit-Limit: 10000
X-RateLimit-Remaining: 4231
X-RateLimit-Reset: 1714000000
X-RateLimit-Plan: pro
```

Free SaaS

If your rules are fully visible, they become a target. A bad actor who knows "10 requests per minute per IP" will just rotate IPs

Don't expose:

- Exact limits
- Exact reset times
- Which dimension you're limiting on (IP? user? endpoint?)

```
HTTP/1.1 429 Too Many Requests
Retry-After: 60
X-RateLimit-Plan: standard
```

More things to consider...

Before you 429, consider a 429 warning: return a X-RateLimit-Warning header when a user is at 80% of their quota. Give them a chance to back off before you cut them off. Sophisticated API clients will handle this gracefully. It's a sign of a mature platform.

&&

```
1  {
2    "error": "rate_limit_exceeded",
3    "message": "You have exceeded your request quota. Please retry after 30 seconds.",
4    "retry_after": 30,
5    "docs": "https://docs.yourapi.com/rate-limits"
6  }
```

Observability

- Connected clients: unexpected spikes mean connection pool misconfiguration
- Memory usage: cardinality explosion shows up here first
- Keyspace hits/misses: means TTLs are expiring too early
- Evicted keys: if Redis is evicting your rate limit keys under memory pressure, your limits are silently disappearing. maxmemory-policy should never be allkeys-lru on a rate limiting Redis. Use a dedicated instance or set noeviction.
- Replication lag: on Sentinel, this tells you how stale your replicas are

Observability (Alerts)

1. Rejection rate > X% for Y minutes: you're either under attack or a limit is misconfigured
2. Redis latency p99 > 10ms
3. Circuit breaker transitions to Open
4. Fallback active for > 60 seconds

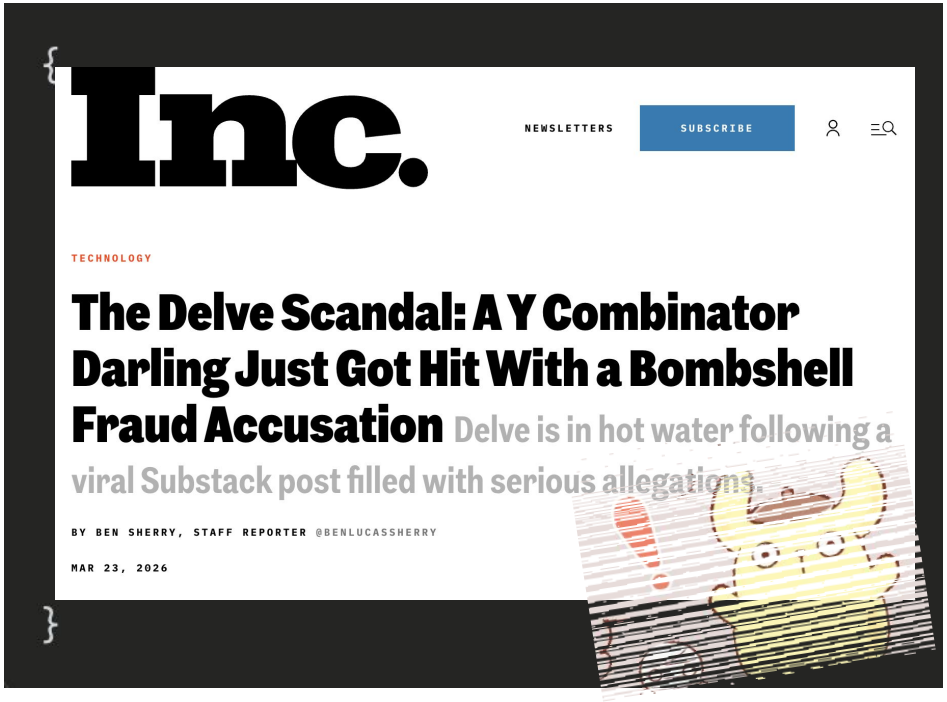
A great log would include the following (don't include ip/user info for compliancy)

```
{
  "event": "rate_limit_exceeded",
  "key": "tenant:acme:user:usr_123",
  "dimension": "user",
  "limit": 1000,
  "window": "60s",
  "current_count": 1001,
  "endpoint": "/v1/messages",
  "timestamp": "2026-04-15T12:00:00Z"
}
```

Observability (Alerts)

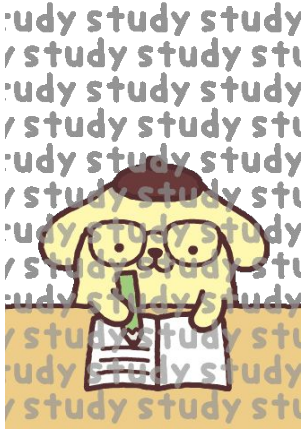
1. Rejection rate > X% for Y minutes: you're either under attack or a limit is misconfigured
2. Redis latency p99 > 10ms
3. Circuit breaker transitions to Open
4. Fallback active for > 60 seconds

A great log would include the following (don't include ip/user info for compliancy)



Key Takeaways:

1. Rate limiting exists to protect infrastructure, ensure fairness, and make systems predictable: Healthcare.gov learned this the hard way
2. Naive implementations have many issues that only appear in production, under load, at the worst possible time.
3. When Redis dies, you need a plan you made before the outage, not during it
4. Key design is an afterthought until cardinality explodes or a hotspot key becomes your bottleneck
5. A 429 is a communication: how you respond to it reflects the maturity of your platform



THANKS!

and enjoy the rest of MTL_Code!

Socials:
- linkedin.com/in/timothy-klint
- github.com/tjklint
- tjklint.com